

Smart Eyes

Meta Wearables (DAT) + Gemini Visual Inspection Prototype

Research / Testing-Track Application: Technical Documentation

Client:	Swiss Safety Center AG
Author:	Teodor Glisic
Lecturer:	Prof. Dr. Yannick Suter
City, Date:	Olten, 2026

Preface

This is client-facing technical documentation for the Smart Eyes research/testing-track prototype, one of two applications in the [smart_eyes_applications](#) repository (the other, [SmartEyesMVP](#), is the later use-case-driven client build). Following the documentation brief, no source code is embedded here; each component is instead linked to its file in the repository. Setup, permission, and architecture explanations follow Meta's official [iOS integration guide](#) for the Wearables Device Access Toolkit (DAT).

Sanitized handoff copy: All developer-specific credentials (Meta ClientToken / MetaAppID, Apple Developer Team, Firebase configuration) have been removed or replaced with placeholders. See [SETUP.md](#) for exactly what must be supplied before the project will build.

Table of Contents

Preface.....	II
Table of Contents.....	III
1 Project Overview	1
1.1 What the app does, in plain terms	1
1.2 Which Meta DAT SDK features it uses	2
2 Codebase Explanation	3
2.1 Project structure	3
2.2 Architecture at a glance.....	4
2.3 Key Files	5
2.4 How the Meta DAT SDK is implemented	6
2.5 How state is managed.....	6
2.6 Permissions and registration flow	7
2.7 How the Gemini integration works	7
2.8 What gets logged	8
2.9 Screenshots	9
3 Prerequisites & Setup.....	11
3.1 Tooling and hardware.....	11
3.2 Swift Package Manager dependencies	11
3.3 Info.plist and build-settings configuration.....	12
3.4 Credentials to supply (three places)	13
4 Step-by-Step Replication Guide	14
4.1 Part A — Prepare the glasses and accounts	14
4.2 Part B — Recreate and configure the app	14
4.3 Part C — Build, register, and verify	15
5 Extending the Project	16
5.1 Adding another Gemini model	16

5.2	Adding a new inspection phase	17
5.3	Example: a complete phase entry	18
6	Known Rough Edges.....	19
7	Distribution & Signing Validity	20
	References.....	21

1 Project Overview

Smart Eyes is an iPhone app that turns a pair of Meta smart glasses into a hands-free visual-inspection assistant. The wearer looks at something — a pressure gauge, a valve, a pipe joint, a piece of equipment — and the glasses' camera view is streamed to the paired iPhone. With a spoken command or a tap, the app captures a high-resolution photo, sends it to Google's Gemini vision model, and reads the answer back aloud. In effect, the user can ask “what am I looking at, and is anything wrong with it?”.

This particular build is the research and testing-track version created for a bachelor thesis. Its job is not to be a polished product but to prove and measure the full pipeline end to end: Meta glasses → iPhone capture → Gemini analysis → spoken / written report, with every trial recorded for later evaluation. To support that, it offers three selectable inspection phases of increasing difficulty, and an audit log that scores and exports each result.

1.1 What the app does, in plain terms

- Pairs with Meta smart glasses and shows a live low-resolution preview of what the wearer sees.
- Captures a full-resolution photo on demand — by the buttons in the application, or hands-free by saying the wake word “inspect”, though during testing this functionality was unstable, since a persistent listening stream would be opened, waiting for the trigger word.
- More precisely, the Inspection tab offers three separate capture/test actions, not just one: analyze the current high-resolution photo (the main path, and what the wake word triggers), analyze the current low-resolution live stream frame directly — skipping high-res capture entirely, useful for quick bandwidth/quality comparisons — and a capture-only action that saves a high-resolution photo straight to the Photos Library without calling Gemini at all, for verifying the glasses' camera independently of the AI pipeline.
- Sends that photo to Gemini and displays a written inspection result, which can also be spoken aloud.
- Can run three test phases — Baseline (everyday objects), Industrial (structured machinery/gauge/valve inspection reports), and Stress Test (degraded, blurred, or obstructed views).
- Logs every trial (image size, prompt, response, latency, token counts) and lets the tester rate accuracy and export the whole log as CSV for analysis.

VoiceTriggerManager keeps AVAudioEngine and SFSpeechRecognizer listening continuously so it can catch the wake word at any moment; in doing so, it holds the iPhone's microphone and audio

session open on the same Bluetooth radio that the DAT SDK is simultaneously using to hold the glasses connection stable and stream frames. That continuous microphone stream competes for — and throttles — the Bluetooth link to the glasses, degrading the live feed and, in the worst case, destabilizing the session entirely. This is the real, underlying cause behind the instability noted above, and it is the reason the MVP build replaced the wake word with an explicit tap-to-ask control instead: its microphone only opens briefly, on demand, after a photo has already been captured — after the bandwidth-sensitive streaming/capture sequence has already completed — rather than being held open continuously in parallel with the glasses connection. This build keeps the wake word as-is because it is part of the empirical record the thesis evaluated, not because it was judged production-ready.

1.2 Which Meta DAT SDK features it uses

The app is built on the [Meta Wearables Device Access Toolkit \(DAT\)](#) for iOS and exercises the SDK's core capabilities: one-time SDK configuration, app-to-glasses **registration** via the Meta AI companion app, continuous **device discovery**, **device sessions**, live **camera streaming** at a chosen resolution and frame rate, and out-of-band **high-resolution photo capture**. Gemini access is handled separately through Firebase AI Logic, so no AI API key is ever stored in the app.

2 Codebase Explanation

The project is a standard SwiftUI iOS app organized into a small App shell, a set of manager singletons that each wrap one external capability, plain data models, and the SwiftUI views. All paths below link directly to the files in the repository.

2.1 Project structure

The repository folder is laid out as a single Xcode project with the Swift sources grouped by responsibility — App shell, Managers, Models, Views, and Support:

SmartEyesV1-research/

└─ SmartEyesPrototypeV1/	App sources
└─ App/	
└─ SmartEyesApp.swift	Entry point, SDK + Firebase startup
└─ ContentView.swift	Tab shell (Inspection · Audit Log)
└─ Managers/	One singleton per external capability
└─ WearableManager.swift	Meta DAT SDK: register/stream/capture
└─ GeminiManager.swift	Gemini via Firebase AI Logic
└─ VoiceTriggerManager.swift	Wake-word listener ("inspect")
└─ SpeechManager.swift	Text-to-speech playback
└─ TelemetryManager.swift	Trial logging + CSV export
└─ Models/	
└─ PhaseCatalog.swift	Phase 1-3 prompt framework
└─ GeminiModelCatalog.swift	Selectable model list
└─ ChatLogStore.swift	Shared chat/audit transcript
└─ Views/	
└─ InspectionView.swift	Live preview, phase, capture
└─ AuditLogView.swift	Past trials, ratings, export
└─ ApiKeySettingsView.swift	Model-selection sheet
└─ Support/	
└─ Shared.swift	Shared UI helpers
└─ Theme.swift	Colors / styling
└─ Assets.xcassets/	App icon, accent color
└─ Info.plist	MWDAT config, URL scheme, bg modes
└─ GoogleService-Info.plist	(you supply – git-ignored)
└─ SmartEyesPrototypeV1.xcodeproj/	Xcode project + SPM resolution
└─ SETUP.md	Credentials you must supply
└─ README.md	In-repo technical reference

2.2 Architecture at a glance

The app follows a lightweight MVVM-style pattern. Each external capability — the glasses, Gemini, speech synthesis, voice triggering, telemetry — is isolated behind a single *@MainActor Observable-Object* singleton in the [Managers](#) folder. SwiftUI views observe these managers and re-render as their *@Published* properties change; the views hold no business logic of their own. Shared, reusable data (the phase catalog, the model list, the chat transcript) lives in the [Models](#) folder so it can be edited in one place.

The entry point [SmartEyesApp.swift](#) configures Firebase and the Wearables SDK once at launch and routes the Meta registration callback URL into the glasses manager. [ContentView.swift](#) wires the two tabs together — [InspectionView.swift](#) (live preview, phase selection, capture) and [Audit-LogView.swift](#) (past trials, ratings, export) — and hosts the shared model-settings sheet [ApiKeySettingsView.swift](#). Visual styling lives in [Theme.swift](#) and [Shared.swift](#).

2.3 Key Files

- [SmartEyesApp.swift](#) — App entry point. Calls Firebase and Wearables SDK configuration at startup and forwards the Meta AI companion app's registration callback URL to the glasses manager.
- [WearableManager.swift](#) — The heart of the DAT integration. Wraps registration, device discovery, session lifecycle, camera streaming, and photo capture; also contains the simulator mock-mode fallback.
- [GeminiManager.swift](#) — Wraps Firebase AI Logic (Gemini). Runs single-shot image-plus-prompt analysis per phase and captures token-usage telemetry from each response.
- [VoiceTriggerManager.swift](#) — Hands-free wake-word listener. Continuously transcribes speech on-device and fires an analysis trigger when it hears “inspect,” with a cooldown to prevent double-firing.
- [SpeechManager.swift](#) — Local text-to-speech playback of analysis results, with markdown stripped for cleaner narration.
- [TelemetryManager.swift](#) — Persists every trial to local JSON, stores user feedback (correct/incorrect, 1–5 stars), and exports the full log as CSV.
- [PhaseCatalog.swift](#) — The Phase 1–3 evaluation framework as data: each phase's name, system instruction, default question, and simulator mock responses. Adding a phase touches only this file.
- [GeminiModelCatalog.swift](#) — The selectable Gemini/Gemma model list shown in settings and passed straight to Firebase AI Logic.
- [ChatLogStore.swift](#) — The chat/audit transcript shared between the Inspection and Audit Log tabs.
- [Info.plist](#) — Background modes, the custom URL scheme, external-accessory protocol, and the MWDAT SDK configuration block.

2.4 How the Meta DAT SDK is implemented

All glasses logic lives in [WearableManager.swift](#), which follows Meta's documented lifecycle end to end. The stages map one-to-one onto the steps in Meta's [iOS integration guide](#):

1. **Configure** — the SDK is configured exactly once at launch (in the app entry point, and defensively re-checked in the manager).
2. **Register** — the user starts registration, which hands off to the Meta AI companion app; when that app calls back into the smarteyesapp:// URL scheme, the entry point passes the URL to the SDK's URL handler. Registration and paired-device state are observed continuously through the SDK's registration and device streams.
3. **Session** — the manager waits for a device to be discovered, then opens a session for that specific device using a `SpecificDeviceSelector`, and waits for the session to reach its started state before continuing.
4. **Stream** — a camera stream is added with a stream configuration. The app deliberately requests the lowest-bandwidth profile the SDK offers (low resolution, 2 FPS) because smart glasses stream over Bluetooth Classic; pushing more bandwidth starves the higher-priority high-resolution photo capture. Frame, state, and error callbacks are wired to their publishers.
5. **Capture** — a JPEG photo capture triggers an out-of-band high-resolution image over the photo-data publisher, independent of the low-resolution live preview. The captured image is what gets sent to Gemini.

This is a deliberate downgrade rather than the starting design point: earlier iterations ran the live stream richer than 360×640 @ 2 FPS, but the glasses' Bluetooth Classic link has a fixed, small bandwidth budget shared between the continuous live preview and the on-demand high-resolution capture. When the preview was allowed to run any richer, it left too little headroom for the 1080×1440 capture to complete before the SDK's internal watchdog timeout fired, causing captures to silently fail or return corrupted or partial data. Deliberately starving the live preview down to the minimum useful frame rate was the fix — a choppy live view traded for every capture reliably returning a usable high-resolution image.

2.5 How state is managed

Every manager is a singleton exposing observable, published properties that describe its current state — for the glasses manager, that includes registration status, discovered devices, whether a stream is active, the current preview frame, the last captured photo, and any error message. SwiftUI

views bind to these properties, so the UI is always a direct reflection of manager state. The DAT SDK pushes updates through Swift async streams (registration and device state) and listener-token publishers (stream state, frames, errors, photo data); the manager forwards those onto the main actor. User preferences (selected Gemini model, voice-on/off) persist through UserDefaults, and the telemetry log persists as JSON in the app's Documents directory via [TelemetryManager.swift](#).

2.6 Permissions and registration flow

Registration is a deep-link round trip. The user triggers registration in the app; the DAT SDK opens the Meta AI companion app, where the user approves the connection and grants the glasses' camera/microphone permissions; the Meta AI app then calls back into the app through the `smarteysapp://` URL scheme declared in [Info.plist](#), and the app forwards that URL to the SDK to complete registration. Camera access to the glasses stream is requested separately through the SDK's permission API just before streaming begins. On-device iOS permissions (microphone and speech recognition for the wake word, photo-library add for saving captures) are requested at the point of use by [VoiceTriggerManager.swift](#), with their usage-description strings supplied through the project's build settings rather than the Info.plist file itself.

2.7 How the Gemini integration works

[GeminiManager.swift](#) talks to Gemini through Firebase AI Logic rather than a raw API key — the only “do we have a key” check in the app is whether Firebase is configured, and the actual credential lives in `GoogleService-Info.plist` and never touches app code. Each analysis is a single-shot request that sends the captured JPEG plus a phase-specific text prompt; the phase's system instruction comes straight from [PhaseCatalog.swift](#). Every response's token usage (text, image, output, thinking, total) is captured and handed to the telemetry log.

Before every request, the captured UIImage is run through `compressImageForAPI()` in `GeminiManager.swift`, which always re-encodes it as JPEG at 0.95 quality, and would additionally downscale it if either dimension exceeded 1920px. In practice that downscale branch never fires: the two possible sources are the 1080×1440 high-res capture and the 360×640 live stream frame (used as a fallback, or selectable directly via the low-res analyze action), and both are already well under 1920px — so today this function only ever compresses, it never resizes. The 1920px cap looks like guardrail headroom for a future higher-resolution capture source rather than something the current pipeline exercises.

2.8 What gets logged

TelemetryManager.swift stores one TrialRecord per analysis, the same shape used by the MVP build:

```
public struct TrialRecord: Codable, Identifiable {
    public let id: UUID
    public let timestamp: Date
    public let phase: String
    public let source: String
    public let captureMode: String
    public let imageWidth: Double
    public let imageHeight: Double
    public let latencySeconds: Double
    public let customPrompt: String
    public let rawResponseText: String
    public let parsedSeverity: String
    public let aiModel: String?
    public var isCorrect: Bool?
    public var rating1To5: Int?
    public var notes: String?
    public var inputTokens: Int?
    public var textInputTokens: Int?
    public var imageInputTokens: Int?
    public var outputTokens: Int?
    public var totalTokens: Int?
    public var thinkingTokens: Int?
}
```

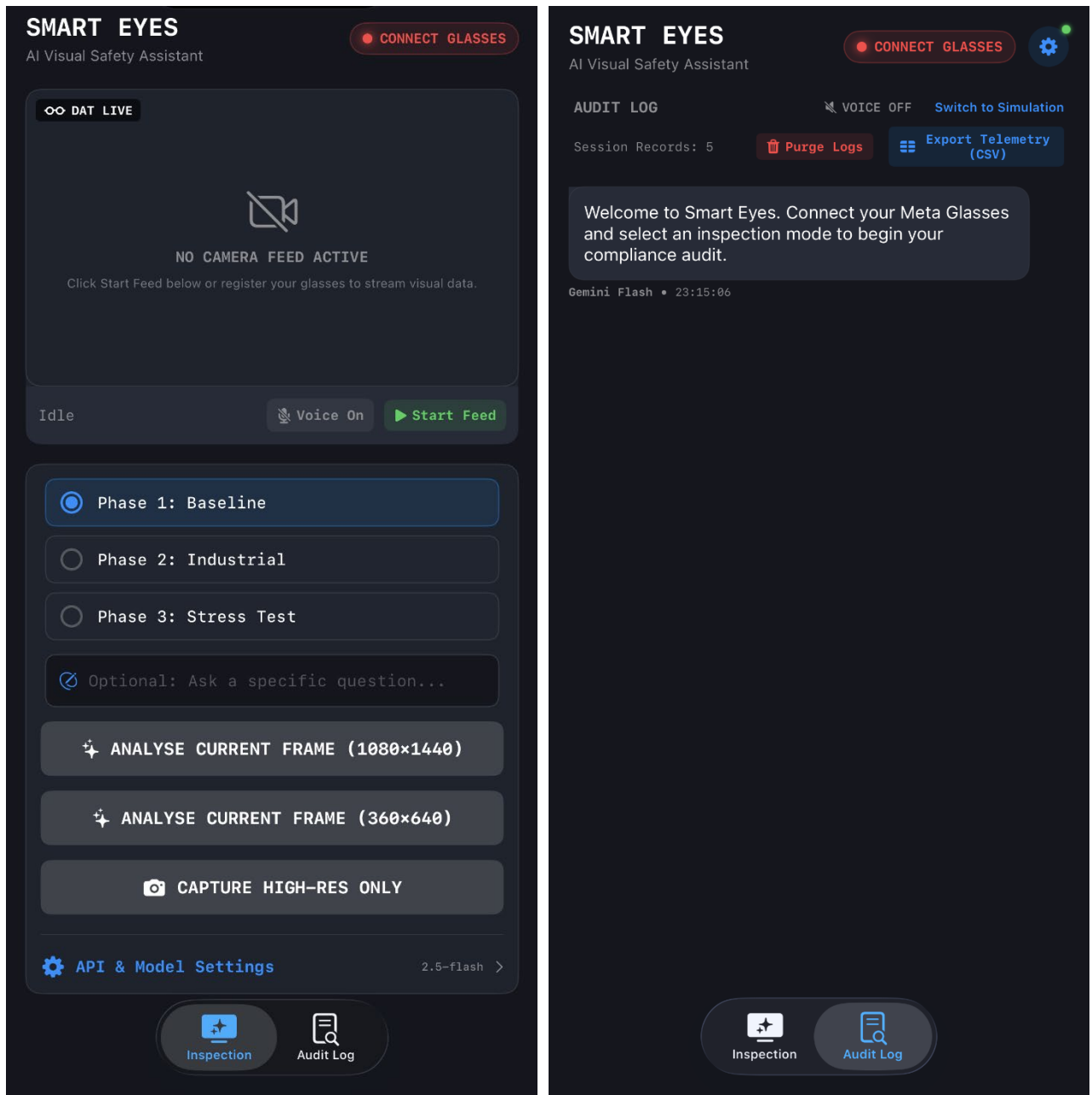
exportCSV() writes one double-semicolon-delimited row per record, with this header:

```
trial_id;;timestamp;;phase;;test_case;;source;;cap-
ture_mode;;image_width;;image_height;;latency_seconds;;custom_prompt;;parsed_se-
verity;;is_correct;;rating;;ai_model;;output;;simulation;;text_input_tokens;;image_in-
put_tokens;;total_input_tokens;;thinking_tokens;;output_tokens;;total_tokens
```

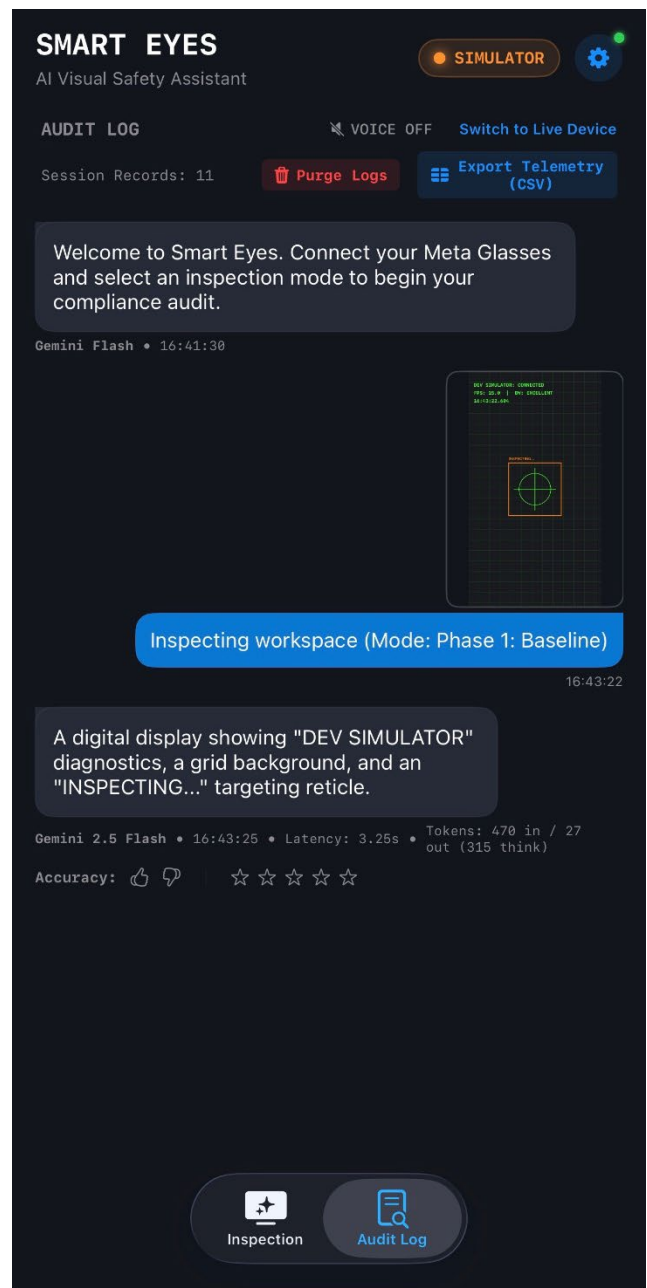
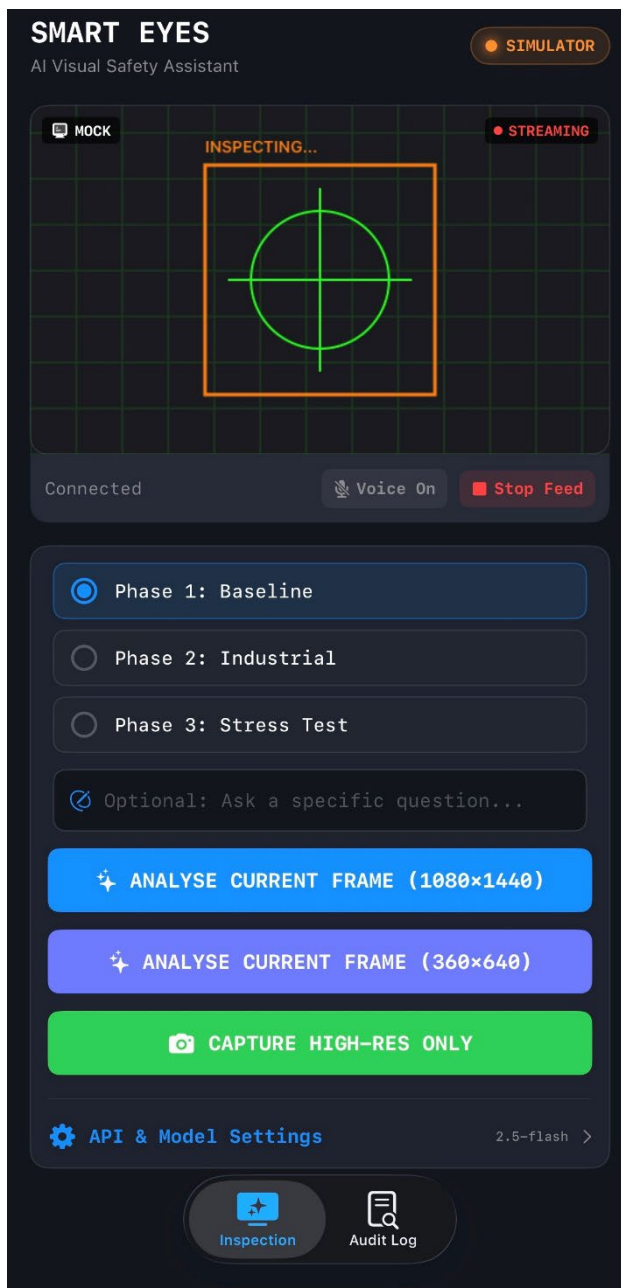
test_case always writes the literal placeholder string “Please enter test case”. That is intentional, not a bug: the tester manually overwrites this text per row in the exported spreadsheet with the actual test case label, so token usage can be analyzed per test case. (The final column is simulation here rather than the MVP's response_status, reflecting whether the row came from a real Gemini call or Simulator mock mode.)

parsedSeverity is not a structured field from Gemini — it's a client-side keyword match over the model's free-text response (InspectionView.triggerInspection): the lowercased answer is checked for “critical”, then “warning”, then “nominal”/“satisfactory”, in that order, defaulting to “N/A” if none match. It's only as reliable as the model's wording, since there's no enum or schema constraining the response.

2.9 Screenshots



Testing-Track Application: Technical Documentation



3 Prerequisites & Setup

Everything below is required before the project will build and run on a physical device. The iOS Simulator can run the app in mock mode without any glasses, Meta credentials, or Firebase configuration, but the real streaming path needs all of it.

Mock mode isn't purely automatic, either: besides auto-enabling in the Simulator, it can be flipped manually at any time from the Audit Log tab's “Switch to Simulation / Switch to Live Device” button (`WearableManager.toggleMockMode()`) — handy for demoing the app without glasses even when running on a physical device.

3.1 Tooling and hardware

- **Xcode:** Version 26.5 (17F42). The project uses a very recent project format (objectVersion 77), so an equivalently recent Xcode is required; Meta officially supports Xcode 14.0+ for the DAT SDK itself.
- **Host macOS:** 26.5.1 (25F80).
- **iOS deployment target:** 26.5 (set in all build configurations). Note this is an unusually high target; a physical test device must run a matching iOS version.
- **Swift language version:** 5.0.
- **Test device:** A physical iPhone. The DAT SDK's Bluetooth/Wi-Fi camera-streaming path does not run in the Simulator.
- **Glasses:** Ray-Ban Meta Wayfarer (Gen 2), paired to the Meta AI companion app with Developer Mode enabled. The DAT SDK also supports Ray-Ban Meta Gen 1, Ray-Ban Meta Optics, and Meta Ray-Ban Display.
- **Accounts:** An Apple Developer Program account (for signing), a Meta Wearables Developer Center account (for the app credentials), and a Firebase project (for Gemini access).

3.2 Swift Package Manager dependencies

The project resolves two packages through Swift Package Manager — no manual download or checkout is needed:

- **Meta Wearables DAT SDK** — github.com/facebook/meta-wearables-dat-ios, resolved at version 0.8.0 (up-to-next-major rule, minimum 0.8.0), providing the MWDATCore and MWDAT-Camera products.

- **Firebase iOS SDK** — github.com/firebase/firebase-ios-sdk, providing the FirebaseAI and FirebaseALLogic products used for Gemini access.

Both packages are public repositories added over HTTPS, so no additional GitHub credentials are needed to resolve them. When you clone this project, Xcode restores both automatically from the committed package resolution.

3.3 Info.plist and build-settings configuration

The DAT SDK is configured through the MWDAT dictionary and supporting keys in [Info.plist](#). The keys present and their roles:

- **MWDAT › AppLinkURLScheme** = smarteyesapp:// — the scheme the Meta AI app uses to call back after registration; it matches the CFBundleURLSchemes entry.
- **MWDAT › MetaAppID** = YOUR_META_APP_ID (placeholder) — replace with your Wearables Developer Center app ID.
- **MWDAT › ClientToken** = YOUR_META_CLIENT_TOKEN (placeholder) — replace with your Wearables Developer Center client token.
- **MWDAT › TeamID** = \$(DEVELOPMENT_TEAM) — populated automatically from your Apple signing team.
- **MWDAT › DAMEnabled** = true — enables the Device Access Toolkit App Model.
- **MWDAT › Analytics › OptOut** = true — this handoff copy opts out of Meta's SDK analytics collection.
- **CFBundleURLTypes** — declares the smarteyesapp URL scheme for the registration call-back.
- **LSApplicationQueriesSchemes** = fb-viewapp, meta-view — allows the app to reach the Meta companion apps.
- **UIBackgroundModes** = bluetooth-central, bluetooth-peripheral, external-accessory — keeps the glasses connection alive in the background.
- **UISupportedExternalAccessoryProtocols** = com.meta.ar.wearable — the External Accessory protocol the SDK communicates over.

The iOS permission usage-description strings are set in the target's build settings (as INFOPLIST_KEY_* entries) rather than in the Info.plist file: camera, microphone, speech recognition, local network, and photo-library-add descriptions are all present there.

3.4 Credentials to supply (three places)

Full step-by-step instructions are in [SETUP.md](#). In summary:

1. **Firestore (Gemini access).** Create a Firestore project, enable AI Logic with the Gemini Developer API, add an iOS app using the project's bundle ID (ch.glisic.SmartEyesPrototypeV1), download the generated GoogleService-Info.plist, and place it next to the app source. This file is git-ignored and never committed.
2. **Meta Wearables Developer Center.** Register your own app to obtain a MetaAppID and ClientToken, then paste them over the two placeholders in Info.plist.
3. **Apple code signing.** The development team is blank in every build configuration; in Xcode, select the target → Signing & Capabilities → choose your Apple Developer Team. Xcode fills in the team ID, which also flows into the MWDAT TeamID key.

4 Step-by-Step Replication Guide

This guide recreates the project from scratch on a fresh machine, in order. It assumes the tooling in Section 3 is installed. Steps 1–4 prepare the glasses and accounts (following Meta's [Setup](#) guide); Steps 5–10 build and configure the app; Steps 11–13 run and verify it.

4.1 Part A — Prepare the glasses and accounts

1. **Update and pair the glasses:** Install the Meta AI companion app on the test iPhone, pair your Ray-Ban Meta Wayfarer (Gen 2) glasses to it, and confirm both the Meta AI app and the glasses firmware meet the DAT version requirements. To check the glasses version: Meta AI app → Devices tab → your device → gear icon → General → About → Version, and update if it is below the minimum.
2. **Enable Developer Mode in the Meta AI app:** Go to Settings → App Info and tap the App version number five times to reveal the Developer Mode toggle; enable it and confirm. Developer Mode lets your unpublished app register and interact with the glasses without submitting it for review. Registered apps appear under Meta AI settings → App connections → Developer mode apps.
3. **Installing the DAT onto the glasses:** After activating the developer mode, you will be prompted to install the latest DAT onto the glasses, which is mandatory, so that the application can communicate with the glasses and vice-versa.
4. **Register the app in the Wearables Developer Center:** Create a Wearables Developer Center account and register an app to obtain a MetaAppID and ClientToken. Keep these for Step 7. (These identifiers are also used for app attestation; attestation is skipped in Developer Mode, but incorrect identifiers will cause connection errors.)

4.2 Part B — Recreate and configure the app

5. **Get the project:** Clone the repository and open the research-track project.
https://github.com/teodorglisc/smart_eyes_applications
6. **Add the Swift Package dependencies:** If starting a project from scratch, in Xcode choose File → Add Package Dependencies..., add github.com/facebook/meta-wearables-dat-ios (version 0.8.0, up-to-next-major) and link its MWDATCore and MWDATCamera products to the app target, then add github.com/firebase/firebase-ios-sdk and link FirebaseAI and FirebaseAILogic. When cloning the existing project, Xcode resolves these automatically from the committed package resolution.

7. **Configure Info.plist and permission strings:** Ensure the MWDAT block, the smarteyesapp URL scheme, the background modes, and the External Accessory protocol are present (see Section 3.3), and that the camera, microphone, speech-recognition, local-network, and photo-library-add usage descriptions are set in the target's build settings.
8. **Enter your Meta credentials:** Replace YOUR_META_APP_ID and YOUR_META_CLIENT_TOKEN in Info.plist with the values from Step 3.
9. **Set up Firebase for Gemini:** In the Firebase console, create/select a project, enable AI Logic → Gemini Developer API, add an iOS app with bundle ID ch.glisic.SmartEyesPrototypeV1, download GoogleService-Info.plist, and add it to the app (next to the app entry point). Confirm Firebase is configured at startup — it already is in the entry point.
10. **Select your signing team:** Target → Signing & Capabilities → choose your Apple Developer Team so DEVELOPMENT_TEAM (and the MWDAT TeamID) is populated.

4.3 Part C — Build, register, and verify

11. **Build and run on the physical iPhone:** Select your device and run.
12. **Register the glasses from the app:** With the glasses on and nearby, tap Register in the app. You will be handed off to the Meta AI app to approve the connection and grant camera/microphone access, then bounced back automatically via the smarteyesapp://callback. The app's registration state should switch to registered and the device should appear as discovered.
13. **Run an inspection and confirm the pipeline:** Once connected, the live low-resolution preview appears. Choose a phase (Baseline, Industrial, or Stress Test), then capture a frame either with the button in the application or hands-free by saying “inspect.” Gemini's written result should appear in the chat log alongside the captured image, and — if voice output is enabled — be read aloud. Open the Audit Log tab to rate accuracy and export the telemetry as CSV, confirming the full glasses → Gemini → report loop works end to end.

5 Extending the Project

The app is deliberately data-driven: the selectable Gemini models and the inspection phases each live in a single catalog file, so extending either is a one-file change with no edits to the networking or UI code.

5.1 Adding another Gemini model

The model picker in the settings sheet is built directly from the list in [GeminiModelCatalog.swift](#). To offer a new model:

1. **Add one entry to the list:** In `GeminiModelCatalog.swift`, append a new model to the `GeminiModel.all` array with two fields: `id` — the exact model name Firebase AI Logic expects (for example `gemini-3.5-flash`) — and `displayName`, the human-readable label shown in the picker.
2. **Keep the id exact:** The `id` is persisted to `UserDefaults` and passed straight to Firebase AI Logic's `generativeModel(modelName:)`; a typo or a name your Firebase project can't serve results in a 404 at request time, not a compile error. Confirm the model is enabled for your Firebase/Gemini project first.
3. **(Optional) add a friendly export label:** The telemetry model-name formatter in `TelemetryManager.swift` maps a few known ids to prettier names for the CSV export; add a case there if you want the new model to read nicely in exported data. This is cosmetic — unmapped ids still export correctly.

Nothing else needs to change: the settings picker in [ApiKeySettingsView.swift](#) iterates the catalog automatically, and the first entry in the list is used as the default on a fresh install.

5.2 Adding a new inspection phase

The Phase 1–3 evaluation framework is defined as data in [PhaseCatalog.swift](#). Each phase is one Phase value; the manager and the views simply read the fields of whichever phase is selected, so adding a Phase 4 (or a new domain-specific phase) touches only this file:

1. **Define the phase:** Create a new Phase with an id (a unique short string), a name (shown in the phase picker), a systemInstruction (the Gemini system prompt that defines the phase's behavior and reporting style), a defaultPrompt (the question sent when the user doesn't dictate a custom one), and mockResponses (a few canned strings used in simulator/mock mode when Firebase isn't configured).
2. **Register it:** Add your new phase to the Phase.all array so it appears in the picker.
3. **Test in the Simulator first:** Because mock mode randomly returns one of the phase's mockResponses, you can exercise the new phase's UI end to end in the iOS Simulator before ever putting on the glasses.

That is the entire change. The analysis call in [GeminiManager.swift](#) reads the selected phase's system instruction and prompt directly, and the phase picker is generated from Phase.all — neither needs editing. (Note: the sibling MVP build uses a similar catalog but calls these “use cases” rather than “phases,” and adds an optional Google Search grounding flag per entry.)

5.3 Example: a complete phase entry

For reference, a complete, working phase entry:

```
static let phase1 = Phase(  
    id: "phase1",  
    name: "Phase 1: Baseline",  
    systemInstruction: ""  
    You are a fast, objective visual assistant for smart glasses baseline testing. Your  
    goal is to identify common objects, basic safety features, flat reference materials, and  
    baseline industrial components under various lighting conditions.  
    Guidelines:  
    1. **Extreme Brevity**: Keep your response under 1-2 sentences. State what you see  
    immediately without introductory filler (e.g., avoid "I can see..." or "In this  
    image...").  
    2. **Identification Scope**: Accurately recognize everyday items (e.g., apples),  
    basic room safety elements (e.g., exit doors), printed 2D schematics/manuals, and generic  
    industrial elements (e.g., pipes, pressure gauges) without assuming deep context.  
    3. **Lighting Adaptability**: If the image has extreme lighting (bright sunlight  
    glare or dark shadows), attempt to identify the object but briefly note if the visual  
    quality is degraded (e.g., "A pressure gauge, slightly obscured by direct sunlight  
    glare").  
    "",  
    defaultPrompt: "Identify the baseline objects and features in this image.",  
    mockResponses: [  
        "Red apple on the table, clean and positioned in the center.",  
        "A pressure gauge, slightly obscured by direct sunlight glare.",  
        "Standard industrial exit door, clearly marked with a green exit sign under normal  
    office lighting."  
    ]  
)
```

Appending a new Phase to the Phase.all array (in the same file) is the entire integration — no other file needs to change. mockResponses is what Simulator mock mode returns at random when Fire-base isn't configured, so a new phase is testable in the iOS Simulator before ever touching physical glasses.

6 Known Rough Edges

A few known, low-severity issues are documented here for transparency rather than left implicit:

- Wake-word Bluetooth contention. Already covered above (Chapter 1.1): the hands-free “inspect” trigger is unstable by design trade-off, not by bug — included here as a cross-reference so this list reads as a complete known-issues checklist.

7 Distribution & Signing Validity

Publishing to the App Store is not currently supported for DAT-based apps: Meta's SDK uses Apple's ExternalAccessory framework, which leads to App Store rejection under Apple's MFi program and privacy-manifest requirements (Meta, iOS integration guide). Meta states it plans to support submission in the future; until then, distribution is limited to development installs or Meta's release channels — so this project is installed directly from Xcode onto a physical device. How long that installed build keeps working depends on the Apple Developer account used to sign it.

- **Free Apple Developer account (a standard personal Apple ID):** the provisioning profile expires after 7 days. Once it lapses, launching the app on the device fails or crashes until you reconnect the device to Xcode and re-install / re-sign it.
- **Paid Apple Developer Program account (\$99/year):** a development or ad-hoc build installed via Xcode stays functional for up to 1 year, or until the paid membership expires — whichever comes first.

Notes and alternatives:

- **TestFlight:** with a paid account, builds distributed via TestFlight stay valid for 90 days.

For this project. Development used no paid subscription, so the installed build expires after 7 days — but it can simply be reinstalled / re-loaded from Xcode manually to reset the 7-day window.

References

- [Meta Wearables DAT — iOS integration guide](#)
- [Meta Wearables DAT — Setup / getting started](#)
- [facebook/meta-wearables-dat-ios \(SDK source\)](#)
- [firebase/firebase-ios-sdk](#)
- [Firebase AI Logic documentation](#)
- [Project repository](#)